HT-based Model Equality Testing

Preliminary Progress Report

Zhuoying Zhang

ingrid_ zh@berkeley.edu

July 31, 2025

1. Recap
2. Formulation
3. Experiments
4. Next Steps

1 Recap

2 Formulation

3 Experiments

4 Next Steps

Previous Experiments

We presented initial experiments and open issues

Setup:

- 20 models from 5 families
- Computed alpha tensors and pairwise Spearman correlations

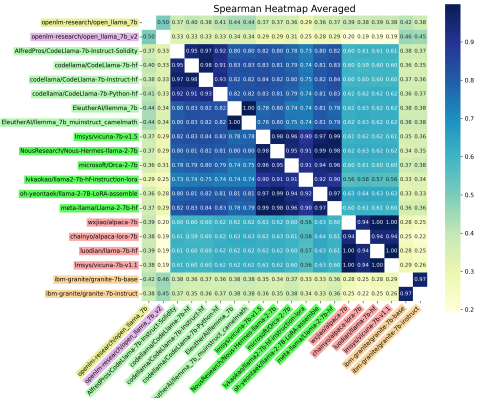


Figure 1: Averaged SR

Issues & Directions

Key Issues Identified:

1. LoRA-tuned models show weak signals in certain modules
2. Sensitivity varies across modules
3. Metric may reflect shared training path, not strict independence
4. Distilled models have mismatched layer counts, hard to compare

Proposed Directions:

- Try subtracting LoRA low-rank components
- Try different fitting strategy to achieve more robust metrics
- Analyze how tensors relate to downstream task performance
- Look into specific fields within tensors
- Distillation lineage needs to be known for proper independence testing

1 Recap

2 Formulation

3 Experiments

4 Next Steps

Encoder

Suppose we have a large language model (LLM) m, and we extract its internal spectral metrics from the weight matrices. These are denoted by a tensor α, obtained from a spectral mapping:

$$\alpha : \mathcal{M} \to W \to \mathrm{Spec}(W) \to \mathcal{T} \tag{1}$$

We treat $\alpha \in \mathcal{T}$ as the structural signature of the model and aim to embed it into a fixed-dimensional space. Specifically, we define an encoder function ϕ_m that maps the tensor to a d-dimensional embedding:

$$\phi_m : \mathcal{T} \to \mathbb{R}^d, \quad \alpha \mapsto v_m \tag{2}$$

This embedding should capture both the models architectural properties and its training-related spectral patterns. Importantly, $\mathcal{T}$ may contain tensors of varying shapes, so ϕ_m must be designed to generalize across architectures.

Encoder

To model family information, we assign each model family $f \in \mathscr{F}$ a learnable embedding:

$$\phi_f : \mathscr{F} \to \mathbb{R}^d, \quad f \mapsto v_f \tag{3}$$

Similarly, for prompt-based analysis, we may learn a representation function for prompts $p \in \mathscr{P}$:

$$\phi_p : \mathscr{P} \to \mathbb{R}^d, \quad p \mapsto v_p \tag{4}$$

Decoder

Independence Tests

Given two models m_1 and m_2, the embeddings enable us to test both the independence between them with the distance $l(m_1, m_2)$ and the family that they belong to.
For one model m_1, we want to test whether it belongs to a family in $\mathscr{F} = \{f_1, f_2, \ldots, f_K\}$.
We train a classifier $\psi_{\mathscr{F}} : \mathbb{R}^d \to \mathscr{F}$ s.t.

$$\psi_{\mathscr{F}}(v_{m_1}) = softmax([\mathbf{w}^\top (v_{m_1} \odot v_{f_1}), \ldots, \mathbf{w}^\top (v_{m_1} \odot v_{f_K})]) \tag{5}$$

where $\mathbf{w}$ is a linear transformation and $\odot$ is the Hadamard product.
Given this property, we can design the distance $l(\cdot, \cdot)$ to be subject to $\mathbf{w}$:

$$l(m_1, m_2) = \mathbf{w}^\top v_{m_1} - \mathbf{w}^\top v_{m_2} \tag{6}$$

Decoder

Behavioral Patterns

To assess task-level performance, we test whether a model can answer a given prompt correctly. We use a similar Hadamard-product structure between the model and prompt embeddings:

$$\varphi(v_m, v_p) = \sigma(\mathbf{u}^\top (v_m \odot v_p) + d) \tag{7}$$

Here, $\mathbf{u} \in \mathbb{R}^d$ is a trainable vector, and d is a bias term. The output represents the probability that model m gives a correct answer to prompt p, as inferred from their latent interaction.

Training Objectives

Family Classification Loss We adopt a multi-class classification setup to predict the family label of each model. For model m, given candidate family embeddings $\{v_{f_1}, \ldots, v_{f_K}\}$, the loss is given by the standard cross-entropy objective:

$$\mathcal{L}_{\text{family}} = -\log\left(\frac{\exp(\mathbf{w}^\top (v_m \odot v_{f_j}))}{\sum_{k=1}^{K} \exp(\mathbf{w}^\top (v_m \odot v_{f_k}))}\right) \tag{8}$$

where f_j is the correct family label of model m.

Prompt Behavioral Loss

We include prompt supervision using observed prompt correctness. We define a mapping $f : \mathcal{M} \times \mathcal{P} \to \{0, 1\}$ to represent the binary correctness $y_{mp} = f(m, p_0)$ for model m and promt $p \in \mathcal{P}$, and use the output of $\varphi(v_m, v_p)$ as in Eq. (7) to define:

$$\mathcal{L}_{\text{prompt}} = -y_{mp} \log \varphi(v_m, v_p) - (1 - y_{mp}) \log(1 - \varphi(v_m, v_p)) \tag{9}$$

Total Loss

The total objective is a weighted combination:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{family}} + \lambda_2 \mathcal{L}_{\text{prompt}} \tag{10}$$

where weights λ_i balance different supervision sources depending on data availability.

Is Spectral α Sufficient?

Given a new model m, suppose we only have access to its spectral tensor α. With the encoder ϕ_m and ϕ_q, we want to see if it is sufficient to predict the model's performance on p_0 with $\varphi(\upsilon_m, \upsilon_{p_0})$. That is to say, we want to test the robustness of

$$\mathbb{E}_D(f(m, p_0)|\alpha) \approx \varphi(\phi_m(\alpha), \upsilon_{p_0}) \tag{11}$$

where $\varphi(\phi_m(\alpha), \upsilon_{p_0})$ is the expected correctness to p_0. This can be transformed into the evaluation of the variance.

$$\mathbb{E}_D(|f(m, p_0) - \varphi(\phi_m(\alpha), \upsilon_{p_0})|^2|\alpha) \tag{12}$$

If the variance is large, then ϕ_m is not a 'good enough' encoder to capture the model's performance. Instead, the embeddings should be directly derived from $f(m, \mathscr{P})$.

1 Recap

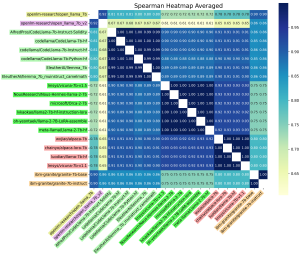
2 Formulation

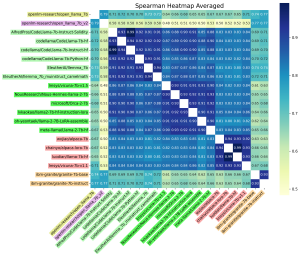
3 Experiments

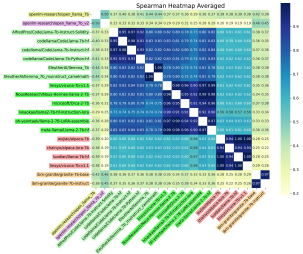
4 Next Steps

Fitting Strategies

- MLE might be unstable → test alternative fitting strategies
- Results: Some separability is retained but not very significant







Prompt-Triggered Fingerprints

Revisited Wuyang's previous work:

- Uses prompts to stimulate attention heads
- Observes how α tensor shifts in response
- Has records of model performance on benchmark datasets

Tried using α (from matrix A) as a fingerprint:

$$A = softmax((QK^\top)/\sqrt{d} \tag{13}$$

where $Q = HW_Q$, $K = HW_K$, H is subject to prompts.

Prompt-Triggered Fingerprints

- Limited run on a few models using the **AQUA** dataset
- Separability exists, but no dramatic improvement

Plan: test on more models and datasets before deciding to adopt

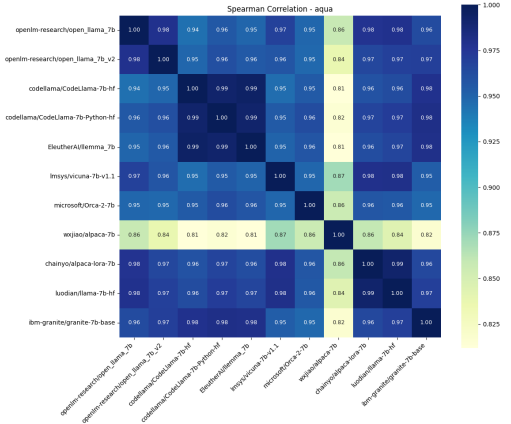


Figure 2: Averaged SR on AQuA

1 Recap

2 Formulation

3 Experiments

4 Next Steps

Next Steps

- **Can the embedding be trained?**
  Explore learning encoders from models/prompts/families into a shared space based on attention features.
- **Use supervision from Wuyang's work**
  Investigate whether their metrics (e.g., function-space similarity) can supervise embedding training.
- **Clarify goal:**
  Are we focusing only on *independence testing*, or is it possible to have a general-purpose embedding that captures both family structure and behavior?

Thank you for listening !

Zhuoying Zhang

ingrid_zh@berkeley.edu